ROBERT v 2.1.0 2026/02/25 19:06:58

How to cite: Dalmau, D.; Alegre Requena, J. V. WIREs Comput Mol Sci. 2024, 14, e1733.

Section A. ROBERT Score

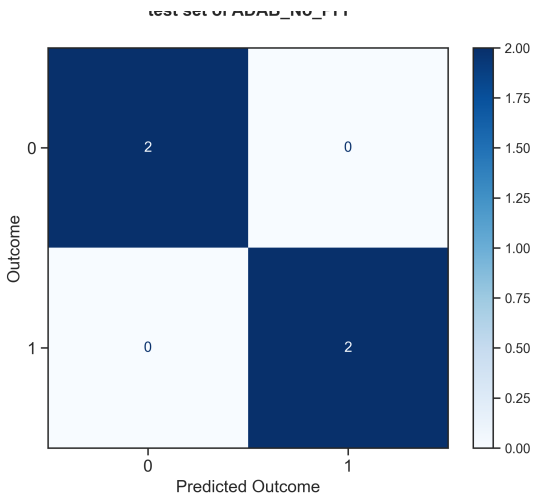
This score is designed to evaluate the models using different metrics.

No PFI (standard descriptor filter) · Score 6

Model = ADAB · CV (train+valid.):Test = 80:20

Points(train+validation):descriptors = 16:3

WEAK



1x 5-fold CV : Accur. = 0.81, F1 score = 0.82, MCC = 0.63

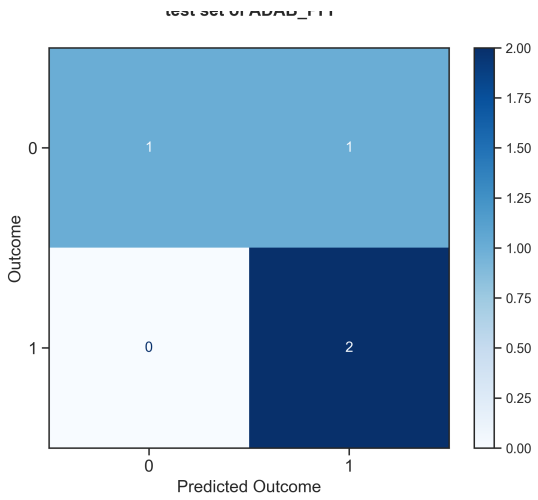
Test : Accur. = 1.0, F1 score = 1.0, MCC = 1.0

PFI (only important descriptors) · Score 7

Model = ADAB · CV (train+valid.):Test = 80:20

Points(train+validation):descriptors = 16:2

MODERATE



1x 5-fold CV : Accur. = 0.81, F1 score = 0.82, MCC = 0.63

Test : Accur. = 0.75, F1 score = 0.8, MCC = 0.58

Severe warnings

- No severe warnings detected

Moderate warnings

- Imprecise predictions (Section B.3b)

Overall assessment

- Moderate model, with important limitations

Severe warnings

- No severe warnings detected

Moderate warnings

- No moderate warnings detected

Overall assessment

- Decent model, but it has limitations

Section B. Advanced Score Analysis

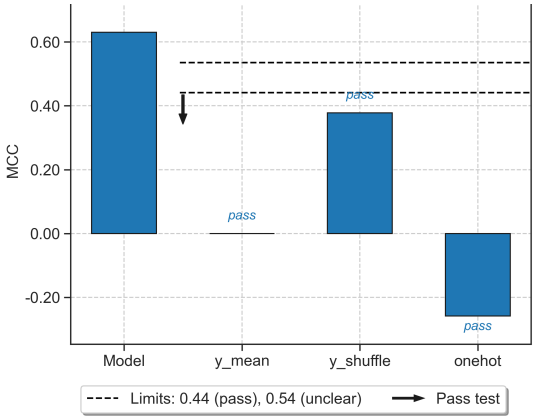
This section explains each component that comprises the ROBERT score. More details here.

1. Model vs "flawed" models (0 / 0)

The model predicts right for the right reasons.

· Scoring from -6 to 0 ·

Pass: 0, Unclear: -1, Fail: -2.

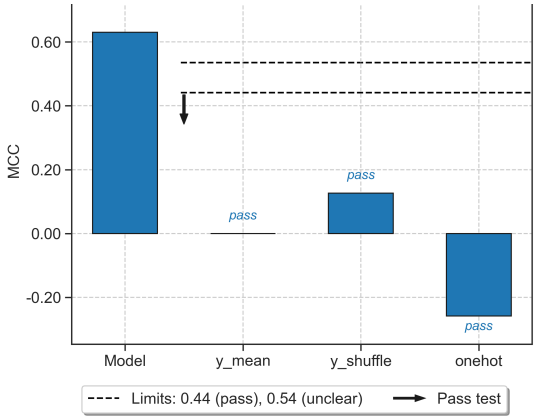


1. Model vs "flawed" models (0 / 0)

The model predicts right for the right reasons.

· Scoring from -6 to 0 ·

Pass: 0, Unclear: -1, Fail: -2.



2. CV predictions of the model (2 / 3)

MCC (10x 5-fold CV) = 0.63.

· Scoring from 0 to 3 ·

MCC >0.75: +3; 0.50-0.75: +2; 0.30-0.50: +1

2. CV predictions of the model (2 / 3)

MCC (10x 5-fold CV) = 0.63.

· Scoring from 0 to 3 ·

MCC >0.75: +3; 0.50-0.75: +2; 0.30-0.50: +1

3. Predictive ability & overfitting

3a. Predictions test set (3 / 3)

MCC (test set) = 1.0.

· Scoring from 0 to 3 ·

MCC >0.75: +3; 0.50-0.75: +2; 0.30-0.50: +1

3b. Prediction accuracy test vs CV (0 / 2)

Relative differences in values from sections 2 and 3a.

The ΔMCC between CV and test is 0.37.

· Scoring from 0 to 2 ·

ΔMCC ≤ 0.15: +2, ΔMCC ≤ 0.30: +1

3. Predictive ability & overfitting

3a. Predictions test set (2 / 3)

MCC (test set) = 0.58.

· Scoring from 0 to 3 ·

MCC >0.75: +3; 0.50-0.75: +2; 0.30-0.50: +1

3b. Prediction accuracy test vs CV (2 / 2)

Relative differences in values from sections 2 and 3a.

The ΔMCC between CV and test is 0.05.

· Scoring from 0 to 2 ·

ΔMCC ≤ 0.15: +2, ΔMCC ≤ 0.30: +1

3c. Consistency (sorted CV) (1 / 2)

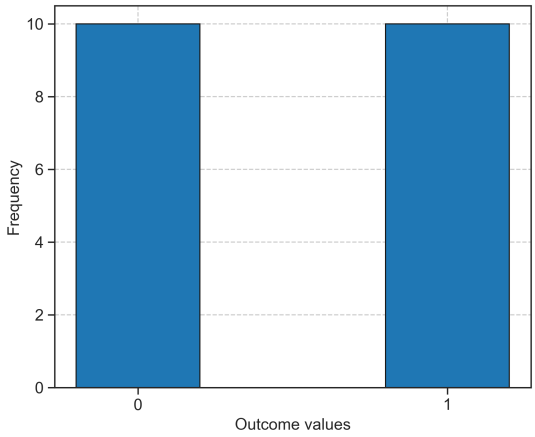
Scaled MCCs across 5-fold CV:
[1.0, 0.0, 1.0, 0.5, 1.0]
· Scoring from 0 to 2 ·
Every two folds with MCCs ≤ 1.25*min MCC: +1.

3c. Consistency (sorted CV) (1 / 2)

Scaled MCCs across 5-fold CV:
[1.0, 0.0, 1.0, 0.5, 1.0]
· Scoring from 0 to 2 ·
Every two folds with MCCs ≤ 1.25*min MCC: +1.

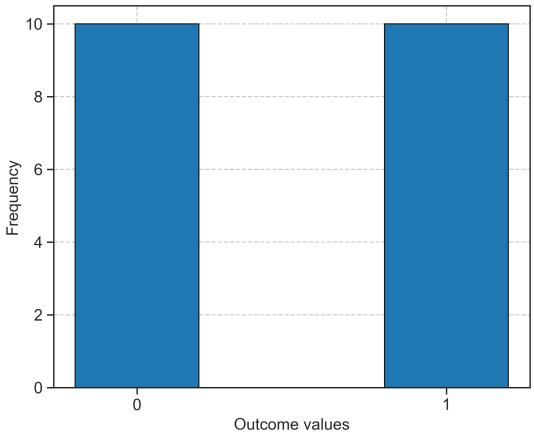
Section C. Distribution of y Values

This section shows the distribution of y values within the training and validation sets.



y distribution analysis

o Your data seems quite uniform

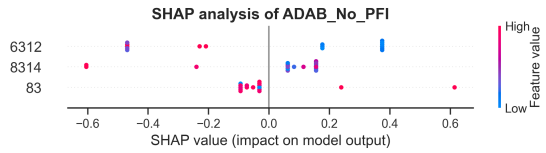


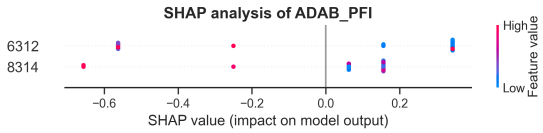
y distribution analysis

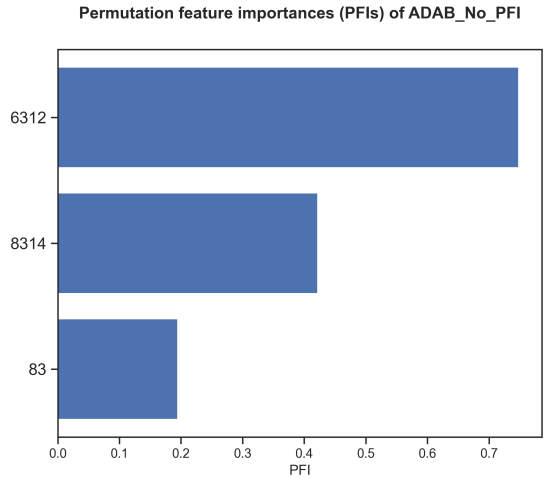
o Your data seems quite uniform

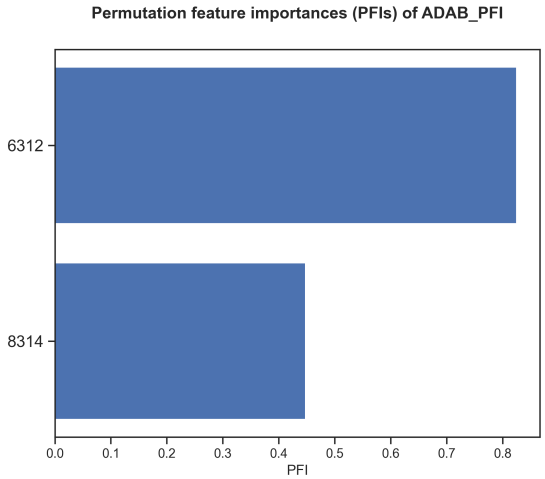
Section D. Feature Importances

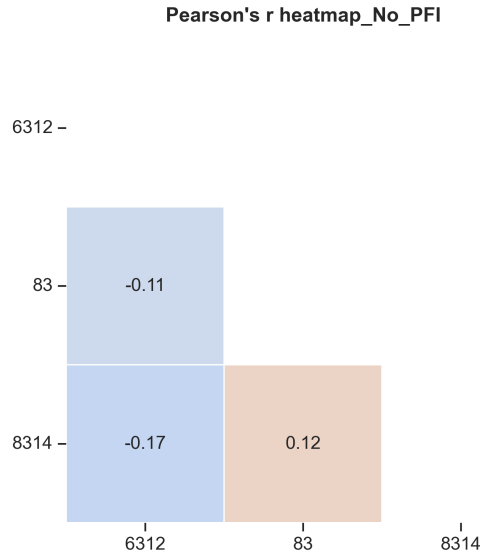
This section presents feature importances measured using the validation set.

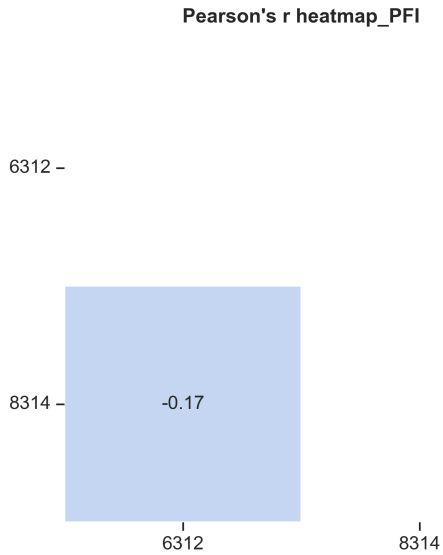












Correlation analysis

- Correlations between variables are acceptable

Correlation analysis

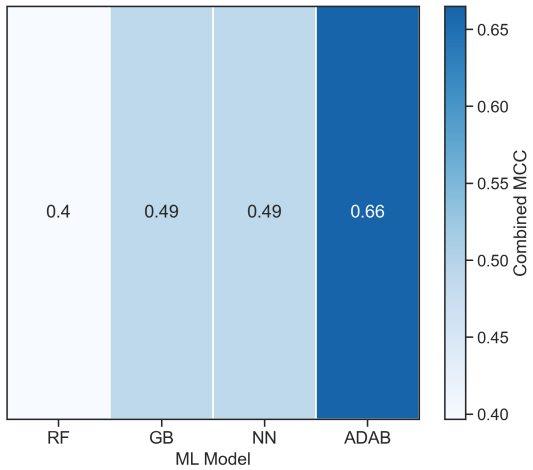
- Correlations between variables are acceptable

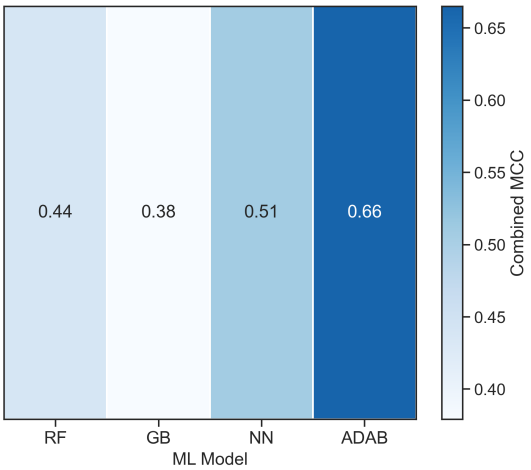
Section E. Outlier Analysis

This feature is disabled in classification problems.

Section F. Model Screening

This section compares different combinations of hyperoptimized algorithms and partition sizes. The combined error is calculated as the product of the training error, validation error, and cross-validation error.





Section G. Reproducibility

This section provides all the instructions to reproduce the results presented.

1. Download these files *(the authors should have uploaded the files as supporting information!)*:

- CSV database (Classification.csv)

2. Install and adjust the versions of the following Python modules:

- Install ROBERT and its dependencies: conda install -y -c conda-forge robert
- Adjust ROBERT version: pip install robert==2.1.0

3. Run ROBERT using this command line in the folder with the CSV database:

python -m robert --csv_name "Classification.csv" --y "Outcome" --names "Name" --type "clas" --repeat_kfolds "1"

4. Execution time, Python version and OS:

Originally run in Python 3.12.12 using Windows 10.0.19045

Total execution time: 70.47 seconds *(the number of processors should be specified by the user)*

Section H. Transparency

This section contains important parameters used in scikit-learn models and ROBERT.

1. Parameters of the scikit-learn models (same keywords as used in scikit-learn):

No PFI (standard descriptor filter):

sklearn model: AdaBoostClassifier
learning_rate: 1.3753540542751057
n_estimators: 47
random_state: 0

PFI (only important descriptors):

sklearn model: AdaBoostClassifier
learning_rate: 1.3753540542751057
n_estimators: 47
random_state: 0

2. ROBERT options, including prediction type (REG or CLAS), folds and repeats used for CV, etc:

No PFI (standard descriptor filter):

type: clas
kfold: 5
repeat_kfolds: 1
seed: 0
error_type: mcc
split: rnd

PFI (only important descriptors):

type: clas
kfold: 5
repeat_kfolds: 1
seed: 0
error_type: mcc
split: rnd

Section I. Abbreviations

Reference section for the abbreviations used.

ACC: accuracy
ADAB: AdaBoost
CSV: comma separated values
CLAS: classification
CV: cross-validation
F1 score: balanced F-score
GB: gradient boosting
GP: gaussian process
KN: k-nearest neighbors
MAE: root-mean-square error
MCC: Matthew's correl. coefficient
ML: machine learning
MVL: multivariate lineal models
NN: neural network
PFI: permutation feature importance
R2: coefficient of determination
REG: Regression
RF: random forest
RMSE: root mean square error
RND: random
SHAP: Shapley additive explanations
VR: voting regressor

Miscellaneous

General tips to improve the models and instructions to predict new values.

Some general tips to improve the score

1. Adding meaningful datapoints might help to improve the model. Also, using a uniform population of datapoints across the whole range of y values usually helps to obtain reliable predictions across the whole range. More information about the range of y values used is available in Section C.

2. Adding meaningful descriptors or replacing/deleting the least useful descriptors used might help. Feature importances are gathered in Section D.

How to predict new values with these models?

1. Create a CSV database with the new points, including the necessary descriptors.

2. Place the CSV file in the parent folder (i.e., where the module folders were created)

3. Run the PREDICT module as 'python -m robert --predict --csv_test FILENAME.csv'.

4. The predictions will be shown at the end of the resulting PDF report and will be stored in the last column of two CSV files called MODEL_SIZE_test(_No)_PFI.csv, which are in the PREDICT folder.